
C: Como Programar

6ª Edição · Paul Deitel & Harvey Deitel

Capítulo 4

Controle de Programa em C

Exercícios (4.5–4.37)

Resolvidos passo a passo, com código completo

Contents

1	Exercício 4.5 – Identificar e Corrigir Erros	2
2	Exercício 4.6 – Valores impressos por estruturas for	4
3	Exercício 4.7 – Estruturas for para sequências	4
4	Exercício 4.8 – O que o programa faz?	5
5	Exercício 4.9 – Soma de uma Sequência de Inteiros	6
6	Exercício 4.10 – Média de uma Sequência (Sentinela 9999)	6
7	Exercício 4.11 – Achar o Menor	7
8	Exercício 4.12 – Soma dos Pares de 2 a 30	8
9	Exercício 4.13 – Produto dos Ímpares de 1 a 15	8
10	Exercício 4.14 – Tabela de Fatoriais (1 a 5)	10
11	Exercício 4.15 – Juros Compostos com Várias Taxas	11
12	Exercício 4.16 – Quatro Padrões de Triângulos	12
13	Exercício 4.17 – Limites de Crédito Cortados pela Metade	13
14	Exercício 4.18 – Gráfico de Barras (Histograma)	15
15	Exercício 4.19 – Calculando Vendas com switch	15
16	Exercício 4.20 – Tabelas-Verdade	17
17	Exercício 4.21 – Inicialização da Variável de Controle na Declaração	17
18	Exercício 4.22 – Nota Média da Classe	19
19	Exercício 4.24 – Avaliação de Expressões	20
20	Exercício 4.25 – Tabela Decimal/Binário/Octal/Hex	21
21	Exercício 4.26 – Cálculo de π por Série Infinita	21
22	Exercício 4.27 – Triplas de Pitágoras (Força Bruta)	23
23	Exercício 4.28 – Pagamento Semanal com Múltiplos Códigos	24
24	Exercício 4.29 – Leis de De Morgan	26
25	Exercício 4.31 – Losango	27
26	Exercício 4.32 – Losango com Tamanho Variável	28

27 Exercício 4.33 – Equivalentes Romanos de 1 a 100	29
28 Exercício 4.34 – do...while vs. while	31
29 Exercício 4.35 – Removendo break	31
30 Exercício 4.36 – O que faz o segmento de programa?	33
31 Exercício 4.37 – Removendo continue	34

1. Exercício 4.5 – Identificar e Corrigir Erros

Resposta detalhada

a) **Código:** For (x = 100, x >= 1, x++)

Erros: (1) For com F maiúsculo (C é case-sensitive); (2) usa-se *vírgula* em vez de *ponto e vírgula* entre as expressões; (3) com x++ e condição x >= 1, criamos um loop infinito (deveria ser x-).

Correção:

```
1  for ( x = 100; x >= 1; x-- )
2      printf( "%d\n", x );
```

b) **Switch para par/ímpar:** Falta break no case 0.

Correção:

```
1  switch ( valor % 2 ) {
2      case 0:
3          printf( "Inteiro par\n" );
4          break;          /* adiciona break! */
5      case 1:
6          printf( "Inteiro impar\n" );
7          break;
8  }
```

c) **Leitura de inteiro e caractere:** O scanf("%d", &intVal) deixa o caractere '\n' (Enter) no buffer, e getchar captura esse '\n' em vez do 'A'.

Correção: consumir o caractere extra antes do getchar.

```
1  scanf( "%d", &intVal );
2  scanf( " %c", &charVal ); /* o espaço antes de %c
   pula brancos */
3  printf( "Inteiro: %d\nCaractere: %c\n", intVal,
   charVal );
```

d) **Ponto flutuante em for:** O mesmo problema de imprecisão do Exercício 4.4(b). A condição x == .0001 provavelmente nunca será verdadeira exatamente.

Correção: usar inteiro como variável de controle.

e) **Inteiros ímpares de 999 a 1:** x += 2 *aumenta* x, mas a intenção é *diminuir* (de 999 para 1).

Correção:

```
1  for ( x = 999; x >= 1; x -= 2 ) {
2      printf( "%d\n", x );
3  }
```

f) **do...while com erros de capitalização:** Do e While devem ser minúsculos. Além disso, a condição < 100 exclui o 100 (o último par desejado); deve ser <=.

Correção:

```
1 contador = 2;
2 do {
3     if ( contador % 2 == 0 ) {
4         printf( "%d\n", contador );
5     }
6     contador += 2;
7 } while ( contador <= 100 );
```

g) **Soma de 100 a 150:** O ; após) cria um corpo vazio para o for; total += x executa apenas uma vez ao final.

Correção: remover o ;.

```
1 for ( x = 100; x <= 150; x++ ) { /* sem ; depois
2     do ) */
3     total += x;
4 }
```

2. Exercício 4.6 – Valores impressos por estruturas for

Resposta detalhada

- a) `for ( x = 2; x <= 13; x += 2 )` → **2, 4, 6, 8, 10, 12** (não 14, pois $14 > 13$)
- b) `for ( x = 5; x <= 22; x += 7 )` → **5, 12, 19** (próximo seria 26, mas $26 > 22$)
- c) `for ( x = 3; x <= 15; x += 3 )` → **3, 6, 9, 12, 15**
- d) `for ( x = 1; x <= 5; x += 7 )` → **1** apenas (próximo seria 8, mas $8 > 5$)
- e) `for ( x = 12; x >= 2; x -= 3 )` → **12, 9, 6, 3** (próximo seria 0, mas $0 < 2$)

3. Exercício 4.7 – Estruturas for para sequências

Resposta detalhada

- a) **1, 2, 3, 4, 5, 6, 7:**

```
1 for ( i = 1; i <= 7; ++i ) {  
2     printf( "%d ", i );  
3 }
```

- b) **3, 8, 13, 18, 23** (incremento de 5):

```
1 for ( i = 3; i <= 23; i += 5 ) {  
2     printf( "%d ", i );  
3 }
```

- c) **20, 14, 8, 2, -4, -10** (decremento de 6):

```
1 for ( i = 20; i >= -10; i -= 6 ) {  
2     printf( "%d ", i );  
3 }
```

- d) **19, 27, 35, 43, 51** (incremento de 8):

```
1 for ( i = 19; i <= 51; i += 8 ) {  
2     printf( "%d ", i );  
3 }
```

4. Exercício 4.8 – O que o programa faz?

Resposta detalhada

O programa lê dois inteiros **x** e **y** (entre 1 e 20). Em seguida, usa **loops aninhados**: o externo executa **y** vezes (cada iteração imprime uma linha) e o interno executa **x** vezes (imprimindo @ em cada).

Resultado: um **retângulo de arrobas** com **y** linhas e **x** colunas.

Exemplo: se **x** = 5, **y** = 3, a saída será:

Saída do programa

```
@@@@@  
@@@@@  
@@@@@
```

5. Exercício 4.9 – Soma de uma Sequência de Inteiros

Resposta detalhada

```
1  /* Ex4.9: soma_sequencia.c
2     Le N (primeiro valor), depois N inteiros, e exibe a
3     soma */
4
5  #include <stdio.h>
6
7  int main( void )
8  {
9      int n;          /* numero de valores a somar */
10     int valor;       /* valor lido a cada iteracao */
11     int soma = 0;    /* total acumulado */
12     int i;
13
14     printf( "Digite quantos valores voce quer somar: "
15            );
16     scanf( "%d", &n );
17
18     for ( i = 1; i <= n; ++i ) {
19         printf( "Digite o valor %d: ", i );
20         scanf( "%d", &valor );
21         soma += valor;
22     }
23
24     printf( "A soma e: %d\n", soma );
25     return 0;
26 } /* fim da funcao main */
```

6. Exercício 4.10 – Média de uma Sequência (Sentinela 9999)

Resposta detalhada

```
1  /* Ex4.10: media_sentinela.c
2     Calcula a media usando 9999 como sentinela */
3
4  #include <stdio.h>
5
6  int main( void )
7  {
8      int valor;
9      int soma = 0;
10     int contador = 0;
11     float media;
12
13     printf( "Digite inteiros (9999 para terminar): " )
14           ;
15     scanf( "%d", &valor );
```



```
14
15     while ( valor != 9999 ) {
16         soma += valor;
17         ++contador;
18         scanf( "%d", &valor );
19     }
20
21     if ( contador != 0 ) {
22         media = (float) soma / contador;  /* conversao
23                                           para float */
24         printf( "Media de %d valores: %.2f\n",
25                 contador, media );
26     }
27     else {
28         printf( "Nenhum valor foi informado.\n" );
29     }
30
31     return 0;
32 } /* fim da funcao main */
```

Conversão de tipo (cast): (float) soma converte soma para float antes da divisão, garantindo divisão de ponto flutuante (com casas decimais), em vez de divisão inteira (que truncaria).

7. Exercício 4.11 – Achar o Menor

Resposta detalhada

```
1  /* Ex4.11: menor.c
2     Le N, depois N inteiros, e exibe o menor */
3  #include <stdio.h>
4
5  int main( void )
6  {
7      int n, valor, menor, i;
8
9      printf( "Digite quantos valores comparar: " );
10     scanf( "%d", &n );
11
12     printf( "Digite o 1o valor: " );
13     scanf( "%d", &menor );  /* primeiro valor
14                               inicializa menor */
15
16     for ( i = 2; i <= n; ++i ) {
17         printf( "Digite o %do valor: ", i );
18         scanf( "%d", &valor );
19         if ( valor < menor ) {
20             menor = valor;
21         }
22     }
```

```
21     }
22
23     printf( "O menor valor e: %d\n", menor );
24     return 0;
25 } /* fim da funcao main */
```

8. Exercício 4.12 – Soma dos Pares de 2 a 30

Resposta detalhada

```
1  /* Ex4.12: soma_pares.c
2     Soma os inteiros pares de 2 a 30 */
3  #include <stdio.h>
4
5  int main( void )
6  {
7      int soma = 0;
8      int i;
9
10     for ( i = 2; i <= 30; i += 2 ) {
11         soma += i;
12     }
13
14     printf( "Soma dos pares de 2 a 30: %d\n", soma );
15     return 0;
16 } /* fim da funcao main */
```

Resultado: $2 + 4 + 6 + \dots + 30 = 240$.

9. Exercício 4.13 – Produto dos Ímpares de 1 a 15

Resposta detalhada

```
1  /* Ex4.13: produto_impares.c
2     Produto dos inteiros impares de 1 a 15 */
3  #include <stdio.h>
4
5  int main( void )
6  {
7      int produto = 1; /* atencao: produto inicia em 1,
8                        nao 0 */
9      int i;
10
11     for ( i = 1; i <= 15; i += 2 ) {
12         produto *= i;
13     }
```

```
14     printf( "Produto dos impares de 1 a 15: %d\n",  
15           produto );  
16     return 0;  
17 } /* fim da funcao main */
```

Resultado: $1 \times 3 \times 5 \times 7 \times 9 \times 11 \times 13 \times 15 = 2.027.025$.

Boa prática de programação

Atenção à inicialização: totais começam com 0, mas produtos com 1 (elemento neutro da multiplicação). Inicializar produto com 0 é um erro comum – todo o resultado seria 0!

10. Exercício 4.14 – Tabela de Fatoriais (1 a 5)

Resposta detalhada

```
1  /* Ex4.14: fatoriais.c
2     Imprime tabela de fatoriais de 1 a 5 */
3  #include <stdio.h>
4
5  int main( void )
6  {
7      int n, i;
8      long fatorial;
9
10     printf( "%5s%15s\n", "n", "n!" );
11
12     for ( n = 1; n <= 5; ++n ) {
13         fatorial = 1;
14         for ( i = 1; i <= n; ++i ) {
15             fatorial *= i;
16         }
17         printf( "%5d%15ld\n", n, fatorial );
18     }
19
20     return 0;
21 } /* fim da funcao main */
```

Saída do programa

```
n          n!
1           1
2           2
3           6
4          24
5         120
```

Por que dificuldade em 20!? O fatorial cresce *explosivamente*:

$$20! = 2\,432\,902\,008\,176\,640\,000 \approx 2,4 \times 10^{18}$$

Esse valor ultrapassa o limite de `long` em sistemas de 32 bits ($\approx 2,1 \times 10^9$). Mesmo em sistemas de 64 bits, $21! \approx 5,1 \times 10^{19}$ já excede o limite de `long` ($\approx 9,2 \times 10^{18}$). Para fatoriais grandes, é preciso usar `double` (com perda de precisão para valores exatos) ou bibliotecas de aritmética arbitrária.

11. Exercício 4.15 – Juros Compostos com Várias Taxas

Resposta detalhada

```
1  /* Ex4.15: juros_compostos_taxas.c
2     Tabela de juros compostos para taxas de 5% a 10% */
3  #include <stdio.h>
4  #include <math.h>
5
6  int main( void )
7  {
8     double principal = 1000.00;  /* valor inicial */
9     double valor;
10    int taxa, ano;
11
12    for ( taxa = 5; taxa <= 10; ++taxa ) {
13        printf( "\n=== Taxa de %d%% ===\n", taxa );
14        printf( "%5s%21s\n", "Ano", "Valor acumulado"
15                );
16
17        for ( ano = 1; ano <= 10; ++ano ) {
18            valor = principal * pow( 1.0 + taxa /
19                                    100.0, ano );
20            printf( "%5d%21.2f\n", ano, valor );
21        }
22    }
23    return 0;
24 } /* fim da funcao main */
```

Fórmula de juros compostos: $V = P \cdot (1 + r)^n$, onde P é o principal, r a taxa em decimal e n o número de anos.

12. Exercício 4.16 – Quatro Padrões de Triângulos

Resposta detalhada

```
1  /* Ex4.16: triangulos.c
2     Imprime 4 padroes triangulares com loops for
       aninhados */
3  #include <stdio.h>
4
5  int main( void )
6  {
7     int linha, j;
8
9     /* (A) Triangulo crescente alinhado a esquerda */
10    printf( "(A)\n" );
11    for ( linha = 1; linha <= 10; ++linha ) {
12        for ( j = 1; j <= linha; ++j ) {
13            printf( "*" );
14        }
15        printf( "\n" );
16    }
17
18    /* (B) Triangulo decrescente alinhado a esquerda
       */
19    printf( "\n(B)\n" );
20    for ( linha = 10; linha >= 1; --linha ) {
21        for ( j = 1; j <= linha; ++j ) {
22            printf( "*" );
23        }
24        printf( "\n" );
25    }
26
27    /* (C) Triangulo decrescente alinhado a direita */
28    printf( "\n(C)\n" );
29    for ( linha = 10; linha >= 1; --linha ) {
30        for ( j = 1; j <= 10 - linha; ++j ) {
31            printf( " " );    /* espacos antes */
32        }
33        for ( j = 1; j <= linha; ++j ) {
34            printf( "*" );
35        }
36        printf( "\n" );
37    }
38
39    /* (D) Triangulo crescente alinhado a direita */
40    printf( "\n(D)\n" );
41    for ( linha = 1; linha <= 10; ++linha ) {
42        for ( j = 1; j <= 10 - linha; ++j ) {
43            printf( " " );
44        }
```

```
45         for ( j = 1; j <= linha; ++j ) {
46             printf( "*" );
47         }
48         printf( "\n" );
49     }
50
51     return 0;
52 } /* fim da funcao main */
```

13. Exercício 4.17 – Limites de Crédito Cortados pela Metade

Resposta detalhada

```
1  /* Ex4.17: limites_credito_metade.c
2     Analisa 3 clientes apos corte de 50% no limite de
3     credito */
4
5  #include <stdio.h>
6
7  int main( void )
8  {
9      int conta;
10     float limiteAntigo, limiteNovo, saldo;
11     int cliente;
12
13     for ( cliente = 1; cliente <= 3; ++cliente ) {
14         printf( "\n=== Cliente %d ===\n", cliente );
15         printf( "Numero da conta: " );
16         scanf( "%d", &conta );
17
18         printf( "Limite de credito antigo: " );
19         scanf( "%f", &limiteAntigo );
20
21         printf( "Saldo atual: " );
22         scanf( "%f", &saldo );
23
24         limiteNovo = limiteAntigo / 2.0f;
25
26         printf( "Conta: %d\n", conta );
27         printf( "Limite antigo: R$ %.2f\n",
28             limiteAntigo );
29         printf( "Limite novo:    R$ %.2f\n", limiteNovo
30             );
31         printf( "Saldo atual:    R$ %.2f\n", saldo );
32
33         if ( saldo > limiteNovo ) {
34             printf( ">>> Saldo excede o novo limite!
35                 <<<\n" );
36         }
37     }
```

```
32         else {  
33             printf( "Saldo dentro do novo limite.\n" )  
34                 ;  
35         }  
36     }  
37     return 0;  
38 } /* fim da funcao main */
```


14. Exercício 4.18 – Gráfico de Barras (Histograma)

Resposta detalhada

```
1  /* Ex4.18: grafico_barras.c
2     Le 5 numeros (1 a 30) e exibe linha de asteriscos
3     correspondente */
4
5  #include <stdio.h>
6
7  int main( void )
8  {
9      int i, j, valor;
10
11     printf( "Digite 5 numeros entre 1 e 30:\n" );
12
13     for ( i = 1; i <= 5; ++i ) {
14         scanf( "%d", &valor );
15
16         for ( j = 1; j <= valor; ++j ) {
17             printf( "*" );
18         }
19         printf( "\n" );
20     }
21
22     return 0;
23 } /* fim da funcao main */
```

Exemplo de execução:

Saída do programa

```
Digite 5 numeros entre 1 e 30:
3 7 12 5 9
***
*****
*****
*****
*****
```

15. Exercício 4.19 – Calculando Vendas com switch

Resposta detalhada

```
1  /* Ex4.19: vendas_switch.c
2     Calcula valor total de vendas usando switch */
3
4  #include <stdio.h>
5
6  int main( void )
7  {
```

```
7      int produto, quantidade;
8      float preco, total = 0.0f;
9
10     printf( "Digite numero do produto e quantidade
11             vendida\n" );
12     printf( "(produto = -1 para terminar):\n" );
13
14     scanf( "%d", &produto );
15
16     while ( produto != -1 ) {
17         scanf( "%d", &quantidade );
18
19         switch ( produto ) {
20             case 1:
21                 preco = 2.98f;
22                 break;
23             case 2:
24                 preco = 4.50f;
25                 break;
26             case 3:
27                 preco = 9.98f;
28                 break;
29             case 4:
30                 preco = 4.49f;
31                 break;
32             case 5:
33                 preco = 6.87f;
34                 break;
35             default:
36                 preco = 0.0f;
37                 printf( "Produto invalido: %d\n",
38                         produto );
39                 break;
40         }
41
42         total += preco * quantidade;
43
44         printf( "Proximo produto (-1 para terminar): "
45                );
46         scanf( "%d", &produto );
47     }
48
49     printf( "\nValor total de vendas: R$ %.2f\n",
50            total );
51     return 0;
52 } /* fim da funcao main */
```

16. Exercício 4.20 – Tabelas-Verdade

Resposta detalhada

Tabela do operador `&&` (E lógico):

Condição1	Condição2	Cond1 && Cond2
0	0	0
0	não zero	0
não zero	0	0
não zero	não zero	1

Tabela do operador `||` (OU lógico):

Condição1	Condição2	Cond1 Cond2
0	0	0
0	não zero	1
não zero	0	1
não zero	não zero	1

Tabela do operador `!` (NÃO lógico):

Condição1	!Condição1
0	1
não zero	0

Em C: qualquer valor diferente de zero é tratado como “verdadeiro” (representado por 1 na saída de operadores lógicos), e zero é “falso”.

17. Exercício 4.21 – Inicialização da Variável de Controle na Declaração

Resposta detalhada

A Figura 4.2 do livro usa `contador` como variável de controle e o inicializa *dentro* do cabeçalho do `for`. Aqui, fazemos a inicialização junto com a declaração:

```

1  /* Ex4.21: contador na declaracao */
2  #include <stdio.h>
3
4  int main( void )
5  {
6      int contador = 1;  /* inicializa na declaracao */
7  }
```

```
8      for ( ; contador <= 10; ++contador ) { /*  
          primeiro campo vazio */  
9          printf( "%d\n", contador );  
10     }  
11  
12     return 0;  
13 } /* fim da funcao main */
```

Note: no cabeçalho `for`, o primeiro campo (inicialização) está vazio. A inicialização foi feita na declaração da variável.

18. Exercício 4.22 – Nota Média da Classe

Resposta detalhada

```
1  /* Ex4.22: nota_media_classe.c
2     Modificacao do programa de notas-letra para
3         calcular media */
4
5  #include <stdio.h>
6
7  int main( void )
8  {
9      int nota;
10     int total = 0;
11     int contador = 0;
12     float media;
13
14     printf( "Digite as notas (EOF para terminar):\n" )
15     ;
16
17     while ( ( nota = getchar() ) != EOF ) {
18         if ( nota >= '0' && nota <= '9' ) {
19             /* Aqui simplesmente lemos numero por
20                numero
21                via scanf seria mais robusto */
22         }
23     }
24
25     /* Versao simplificada com scanf: */
26     printf( "Digite uma nota (-1 para terminar): " );
27     scanf( "%d", &nota );
28
29     while ( nota != -1 ) {
30         total += nota;
31         ++contador;
32         printf( "Proxima nota (-1 para terminar): " );
33         scanf( "%d", &nota );
34     }
35
36     if ( contador != 0 ) {
37         media = (float) total / contador;
38         printf( "Media da classe: %.2f\n", media );
39     }
40     else {
41         printf( "Nenhuma nota informada.\n" );
42     }
43
44     return 0;
45 } /* fim da funcao main */
```

19. Exercício 4.24 – Avaliação de Expressões

Enunciado 4.24

Considere $i = 1$, $j = 2$, $k = 3$, $m = 2$. Determine o valor impresso por cada `printf`.

Resposta detalhada

Em C, expressões relacionais e lógicas avaliam para **1** (verdadeiro) ou **0** (falso).

- a) $i == 1 \rightarrow 1 == 1$ é **verdadeiro** $\rightarrow$ imprime 1
- b) $j == 3 \rightarrow 2 == 3$ é **falso** $\rightarrow$ imprime 0
- c) $i >= 1 \ \&\& \ j < 4 \rightarrow 1 \geq 1$ (V) E $2 < 4$ (V) $\rightarrow 1$
- d) $m <= 99 \ \&\& \ k < m \rightarrow 2 \leq 99$ (V) E $3 < 2$ (F) $\rightarrow 0$
- e) $j >= i \ || \ k == m \rightarrow 2 \geq 1$ (V) OU $3 == 2$ (F) $\rightarrow 1$
- f) $k+m < j \ || \ 3-j >= k \rightarrow 5 < 2$ (F) OU $1 \geq 3$ (F) $\rightarrow 0$
- g) $!m \rightarrow !2 \rightarrow 0$ (qualquer não-zero $\rightarrow 0$)
- h) $!(j - m) \rightarrow !(2-2) \rightarrow !0 \rightarrow 1$
- i) $!(k > m) \rightarrow !(3 > 2) \rightarrow !1 \rightarrow 0$
- j) $!(j > k) \rightarrow !(2 > 3) \rightarrow !0 \rightarrow 1$

20. Exercício 4.25 – Tabela Decimal/Binário/Octal/Hex

Resposta detalhada

```

1  /* Ex4.25: tabela_bases.c
2     Imprime equivalentes binario, octal e hex de 1 a
3     256 */
4
5  #include <stdio.h>
6
7  int main( void )
8  {
9      int decimal, n, binario, valorPos;
10
11     printf( "%10s%10s%10s%10s\n", "Decimal", "Binario"
12         , "Octal", "Hex" );
13
14     for ( decimal = 1; decimal <= 256; ++decimal ) {
15         /* Calcula representacao binaria como inteiro
16            decimal */
17         binario = 0;
18         valorPos = 1;
19         n = decimal;
20         while ( n > 0 ) {
21             binario += ( n % 2 ) * valorPos;
22             n /= 2;
23             valorPos *= 10;
24         }
25
26         /* Octal e hex impressos com %o e %X (
27            suportados por printf) */
28         printf( "%10d%10d%10o%10X\n", decimal, binario
29             , decimal, decimal );
30     }
31
32     return 0;
33 } /* fim da funcao main */

```

Observação: printf oferece especificadores diretos para octal (%o) e hexadecimal (%X). Para o binário, calculamos manualmente, pois C não tem especificador embutido.

21. Exercício 4.26 – Cálculo de π por Série Infinita

Resposta detalhada

Fórmula (Leibniz): $\pi \approx 4 - \frac{4}{3} + \frac{4}{5} - \frac{4}{7} + \frac{4}{9} - \dots$

```

1  /* Ex4.26: pi_serie.c
2     Aproxima pi pela serie de Leibniz e exibe

```

```

1          convergencaia */
2  #include <stdio.h>
3  #include <math.h>
4
5
6  int main( void )
7  {
8      double pi = 0.0;
9      double termo;
10     int sinal = 1;
11     int n;
12     int denominador;
13
14     printf( "%10s%20s\n", "Termos", "Pi aproximado" );
15
16     for ( n = 1; n <= 100000; ++n ) {
17         denominador = 2 * n - 1;          /* 1, 3, 5, 7,
18             9, ... */
19         termo = 4.0 / denominador;
20         pi += sinal * termo;
21         sinal = -sinal;                  /* alterna o
22             sinal */
23
24         if ( n <= 10 || n == 100 || n == 1000 ||
25             n == 10000 || n == 100000 ) {
26             printf( "%10d%20.6f\n", n, pi );
27         }
28     }
29
30     return 0;
31 } /* fim da funcao main */

```

Análise da convergência (lenta!):

- Após 10 termos: $\pi \approx 3,04$
- Após 100 termos: $\pi \approx 3,13$ (1 casa correta)
- Após 1.000 termos: $\pi \approx 3,1406$ (3 casas)
- Após 10.000 termos: $\pi \approx 3,14149$ (4 casas)
- Após 100.000 termos: $\pi \approx 3,14158$ (5 casas)

Quantos termos para cada precisão?

- Para 3,14: cerca de 200 termos
- Para 3,141: cerca de 1.000 termos
- Para 3,1415: cerca de 10.000 termos
- Para 3,14159: cerca de 100.000 termos

A série de Leibniz é **notoriamente lenta** – cada casa decimal extra exige aproximadamente $10\times$ mais termos!

22. Exercício 4.27 – Triplas de Pitágoras (Força Bruta)

Resposta detalhada

Critério: $\text{cateto1}^2 + \text{cateto2}^2 = \text{hipotenusa}^2$.

```
1  /* Ex4.27: triplas_pitagoras.c
2     Encontra todas as triplas de Pitagoras com lados <=
3         500 */
4
5  #include <stdio.h>
6
7  int main( void )
8  {
9
10     printf( "%6s%6s%6s\n", "Cat1", "Cat2", "Hip" );
11
12     for ( cat1 = 1; cat1 <= 500; ++cat1 ) {
13         for ( cat2 = cat1; cat2 <= 500; ++cat2 ) { /*
14             cat2 >= cat1 */
15             for ( hip = cat2; hip <= 500; ++hip ) {
16                 /* hip >= cat2 */
17                 if ( cat1*cat1 + cat2*cat2 == hip*hip
18                     ) {
19                     printf( "%6d%6d%6d\n", cat1, cat2,
20                         hip );
21                     ++contador;
22                 }
23             }
24         }
25     }
26
27     printf( "\nTotal de triplas encontradas: %d\n",
28         contador );
29     return 0;
30 } /* fim da funcao main */
```

Otimização: para evitar duplicatas (3,4,5 = 4,3,5), exigimos $\text{cat1} \leq \text{cat2} \leq \text{hip}$. As primeiras triplas: (3,4,5), (5,12,13), (6,8,10), (7,24,25), (8,15,17), (9,12,15), (9,40,41), ...

Reflexão sobre força bruta: Este programa testa $500^3 = 125$ milhões de combinações. CPUs modernas executam isso em frações de segundo. Esse exemplo ilustra como o “poder bruto” computacional torna soluções triviais de problemas que antes pareciam inviáveis.

23. Exercício 4.28 – Pagamento Semanal com Múltiplos Códigos

Resposta detalhada

```
1  /* Ex4.28: pagamento_semanal.c
2     Calcula pagamento semanal para 4 tipos de
3     funcionario */
4
5  #include <stdio.h>
6
7  int main( void )
8  {
9      int codigo;
10     float salario, pagamento, vendas, valorPeca;
11     int horas, pecas;
12
13     printf( "Codigos: 1=gerente, 2=horista, 3=comissao
14             , 4=peca\n" );
15     printf( "Codigo do funcionario (-1 para terminar):
16             " );
17     scanf( "%d", &codigo );
18
19     while ( codigo != -1 ) {
20         switch ( codigo ) {
21             case 1: /* Gerente */
22                 printf( "Salario semanal fixo: " );
23                 scanf( "%f", &salario );
24                 pagamento = salario;
25                 break;
26
27             case 2: /* Horista */
28                 printf( "Horas trabalhadas: " );
29                 scanf( "%d", &horas );
30                 printf( "Salario por hora: " );
31                 scanf( "%f", &salario );
32                 if ( horas <= 40 ) {
33                     pagamento = horas * salario;
34                 }
35                 else {
36                     pagamento = 40 * salario +
37                                 ( horas - 40 ) *
38                                 salario * 1.5f;
39                 }
40                 break;
41
42             case 3: /* Comissao */
43                 printf( "Vendas brutas semanais: " );
44                 scanf( "%f", &vendas );
45                 pagamento = 250.0f + 0.057f * vendas;
46                 break;
```

```
43         case 4:  /* Por peca */
44             printf( "Pecas produzidas: " );
45             scanf( "%d", &pecas );
46             printf( "Valor por peca: " );
47             scanf( "%f", &valorPeca );
48             pagamento = pecas * valorPeca;
49             break;
50
51         default:
52             printf( "Codigo invalido!\n" );
53             pagamento = 0.0f;
54             break;
55     }
56
57     printf( "Pagamento semanal: R$ %.2f\n\n",
58           pagamento );
59
60     printf( "Proximo codigo (-1 para terminar): "
61           );
62     scanf( "%d", &codigo );
63
64     return 0;
65 } /* fim da funcao main */
```

24. Exercício 4.29 – Leis de De Morgan

Resposta detalhada

As Leis de De Morgan:

$$\neg(C1 \ \&\& \ C2) \equiv (\neg C1 \ || \ \neg C2)$$

$$\neg(C1 \ || \ C2) \equiv (\neg C1 \ \&\& \ \neg C2)$$

a) Original: $\neg(x < 5) \ \&\& \ \neg(y \geq 7)$

Equivalente: $\neg(x < 5 \ || \ y \geq 7)$ ou $x \geq 5 \ \&\& \ y < 7$

b) Original: $\neg(a == b) \ || \ \neg(g != 5)$

Equivalente: $\neg(a == b \ \&\& \ g != 5)$ ou $a != b \ || \ g == 5$

c) Original: $\neg((x \leq 8) \ \&\& \ (y > 4))$

Equivalente: $\neg(x \leq 8) \ || \ \neg(y > 4)$ ou $x > 8 \ || \ y \leq 4$

d) Original: $\neg((i > 4) \ || \ (j \leq 6))$

Equivalente: $\neg(i > 4) \ \&\& \ \neg(j \leq 6)$ ou $i \leq 4 \ \&\& \ j > 6$

Programa de verificação:

```

1  /* Ex4.29: de_morgan.c
2     Demonstra as Leis de De Morgan */
3  #include <stdio.h>
4
5  int main( void )
6  {
7     int x, y;
8
9     printf( "Digite x e y: " );
10    scanf( "%d%d", &x, &y );
11
12    printf( "\nItem (a):\n" );
13    printf( "    !(x<5) && !(y>=7) = %d\n", !(x<5) && !(
14        y>=7) );
15    printf( "    x>=5 && y<7          = %d\n", x>=5 && y<7
16        );
17
18    /* Repetir para os demais itens */
19
20    return 0;
21 } /* fim da funcao main */

```

25. Exercício 4.31 – Losango

Resposta detalhada

```
1  /* Ex4.31: losango.c
2     Imprime forma de losango com altura fixa */
3  #include <stdio.h>
4
5  int main( void )
6  {
7     int linha, j;
8
9     /* Metade superior: 1, 3, 5, 7, 9 asteriscos */
10    for ( linha = 1; linha <= 5; ++linha ) {
11        for ( j = 1; j <= 5 - linha; ++j ) {
12            printf( " " );
13        }
14        for ( j = 1; j <= 2 * linha - 1; ++j ) {
15            printf( "*" );
16        }
17        printf( "\n" );
18    }
19
20    /* Metade inferior: 7, 5, 3, 1 asteriscos */
21    for ( linha = 4; linha >= 1; --linha ) {
22        for ( j = 1; j <= 5 - linha; ++j ) {
23            printf( " " );
24        }
25        for ( j = 1; j <= 2 * linha - 1; ++j ) {
26            printf( "*" );
27        }
28        printf( "\n" );
29    }
30
31    return 0;
32 } /* fim da funcao main */
```

26. Exercício 4.32 – Losango com Tamanho Variável

Resposta detalhada

```
1  /* Ex4.32: losango_variavel.c
2     Imprime losango cujo numero de linhas e fornecido
3     pelo usuario */
4
5  #include <stdio.h>
6
7  int main( void )
8  {
9      int n;          /* numero impar (1 a 19): linhas
10                      no meio */
11      int meio;       /* (n+1)/2: numero da linha do
12                      meio */
13      int linha, j;
14
15      do {
16          printf( "Digite um numero impar (1 a 19): " );
17          scanf( "%d", &n );
18      } while ( n < 1 || n > 19 || n % 2 == 0 );
19
20      meio = ( n + 1 ) / 2;
21
22      /* Metade superior */
23      for ( linha = 1; linha <= meio; ++linha ) {
24          for ( j = 1; j <= meio - linha; ++j ) {
25              printf( " " );
26          }
27          for ( j = 1; j <= 2 * linha - 1; ++j ) {
28              printf( "*" );
29          }
30          printf( "\n" );
31      }
32
33      /* Metade inferior */
34      for ( linha = meio - 1; linha >= 1; --linha ) {
35          for ( j = 1; j <= meio - linha; ++j ) {
36              printf( " " );
37          }
38          for ( j = 1; j <= 2 * linha - 1; ++j ) {
39              printf( "*" );
40          }
41          printf( "\n" );
42      }
43
44      return 0;
45  } /* fim da funcao main */
```

27. Exercício 4.33 – Equivalentes Romanos de 1 a 100

Resposta detalhada

Algoritmo: usar repetidamente a tabela de valores romanos: I=1, IV=4, V=5, IX=9, X=10, XL=40, L=50, XC=90, C=100. Subtraímos o maior valor possível enquanto adicionamos o símbolo correspondente.

```
1  /* Ex4.33: romanos.c
2      Tabela de equivalentes em algarismos romanos de 1 a
3      100 */
4
5  #include <stdio.h>
6
7  int main( void )
8  {
9      int n, valor;
10
11     printf( "%5s    %s\n", "Dec", "Romano" );
12
13     for ( n = 1; n <= 100; ++n ) {
14         printf( "%5d    ", n );
15         valor = n;
16
17         /* C = 100 */
18         while ( valor >= 100 ) { printf( "C" ); valor
19             -= 100; }
20         /* XC = 90 */
21         while ( valor >= 90 ) { printf( "XC" ); valor
22             -= 90; }
23         /* L = 50 */
24         while ( valor >= 50 ) { printf( "L" ); valor
25             -= 50; }
26         /* XL = 40 */
27         while ( valor >= 40 ) { printf( "XL" ); valor
28             -= 40; }
29         /* X = 10 */
30         while ( valor >= 10 ) { printf( "X" ); valor
31             -= 10; }
32         /* IX = 9 */
33         while ( valor >= 9 ) { printf( "IX" ); valor
34             -= 9; }
35         /* V = 5 */
36         while ( valor >= 5 ) { printf( "V" ); valor
37             -= 5; }
38         /* IV = 4 */
39         while ( valor >= 4 ) { printf( "IV" ); valor
40             -= 4; }
41         /* I = 1 */
42         while ( valor >= 1 ) { printf( "I" ); valor
43             -= 1; }
44     }
```

```
34         printf( "\n" );
35     }
36
37     return 0;
38 } /* fim da funcao main */
```

Exemplos: 14 = XIV, 49 = XLIX, 99 = XCIX, 100 = C.

28. Exercício 4.34 – do...while vs. while

Resposta detalhada

Substituir do...while por while:

A diferença essencial entre as duas estruturas é que **do...while sempre executa o corpo pelo menos uma vez**, enquanto **while** pode não executar nenhuma vez (se a condição já for falsa de início).

Para converter do...while → while: basta executar o corpo *antes* do laço:

```
1  /* Original: do-while */
2  do {
3      /* corpo */
4  } while ( condicao );
5
6  /* Equivalente com while */
7  /* corpo */      /* primeira execucao garantida */
8  while ( condicao ) {
9      /* corpo */ /* repeticao se condicao verdadeira */
10 }
```

Problema ao converter while → do...while: se a condição já for falsa no início, o **while não executa o corpo nem uma vez**, mas o **do...while executa pelo menos uma vez**, o que pode causar comportamento incorreto.

Solução: use uma instrução **if** para envolver o **do...while**:

```
1  if ( condicao ) {
2      do {
3          /* corpo */
4      } while ( condicao );
5  }
```

Assim, se a condição já for falsa, nem o **if** nem o **do...while** executam.

29. Exercício 4.35 – Removendo break

Resposta detalhada

Como remover break de um laço:

A ideia central é introduzir uma **variável de controle adicional** (uma “flag”) que indique se o laço deve continuar. A condição original é estendida com essa flag, unindo as duas formas de saída em uma só (na condição de continuação).

Padrão original com break:

```
1  while ( condicao_principal ) {
2      /* ... */
3      if ( condicao_de_saida ) {
4          break;
5      }
6      /* ... */
```

```
7 }
```

Equivalente sem break:

```
1  int continuar = 1;  /* flag de continuacao */
2  while ( condicao_principal && continuar ) {
3      /* ... */
4      if ( condicao_de_saida ) {
5          continuar = 0;  /* sinaliza saida; loop
6                          terminara apos */
7      }
8      else {
9          /* ... codigo apos o break original ... */
10     }
11 }
```

A divisão do código em if/else garante que, ao detectar a saída, o restante do corpo *não* seja executado nessa iteração. A condição estendida garante que o laço termina na próxima checagem.

30. Exercício 4.36 – O que faz o segmento de programa?

Resposta detalhada

```
1  for ( i = 1; i <= 5; i++ ) {  
2      for ( j = 1; j <= 3; j++ ) {  
3          for ( k = 1; k <= 4; k++ )  
4              printf( "*" );  
5          printf( "\n" );  
6      }  
7      printf( "\n" );  
8  }
```

Análise dos três loops aninhados:

- Loop i (mais externo): executa 5 vezes
- Loop j (médio): para cada i, executa 3 vezes
- Loop k (mais interno): para cada j, imprime 4 asteriscos

Para cada iteração de i, são impressas 3 linhas com 4 asteriscos cada (forma um retângulo 3×4), seguidas de uma linha em branco.

Saída: 5 grupos de retângulos 3×4 de asteriscos, separados por linhas em branco. Total: $5 \times 3 = 15$ linhas com 4 asteriscos, mais 5 linhas em branco.

Saída do programa

```
****  
***  
***  
  
****  
***  
***  
  
****  
***  
***  
  
****  
***  
***  
  
****  
***  
***
```

31. Exercício 4.37 – Removendo continue

Resposta detalhada

Como remover continue de um laço:

O comando `continue` pula o restante da iteração atual e prossegue para a próxima. Para removê-lo, basta colocar o restante do código do laço dentro de um `else` (ou um `if` com a condição negada).

Padrão original com `continue`:

```
1 while ( condicao ) {  
2     /* parte 1 */  
3     if ( condicao_pular ) {  
4         continue;  
5     }  
6     /* parte 2 */  
7 }
```

Equivalente sem `continue`:

```
1 while ( condicao ) {  
2     /* parte 1 */  
3     if ( !condicao_pular ) { /* condicao negada */  
4         /* parte 2 */  
5     }  
6 }
```

Aplicação à Figura 4.12 (programa de soma de 1 a 10 pulando 5):

```
1  /* Original com continue: */  
2  for ( x = 1; x <= 10; ++x ) {  
3      if ( x == 5 ) {  
4          continue;  
5      }  
6      printf( "%d ", x );  
7  }  
8  
9  /* Equivalente sem continue: */  
10 for ( x = 1; x <= 10; ++x ) {  
11     if ( x != 5 ) { /* condicao negada */  
12         printf( "%d ", x );  
13     }  
14 }
```